UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 1: Introduction to Object-Oriented Programming & Java (Part 2)

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

Contents

- **Introduction:** Origins of Java Language
- **Java Basic:**
  - Java API
  - Version
  - Development Tools
- **“Anatomy” of Java Program**

Origins of Java Language

INTRODUCTION

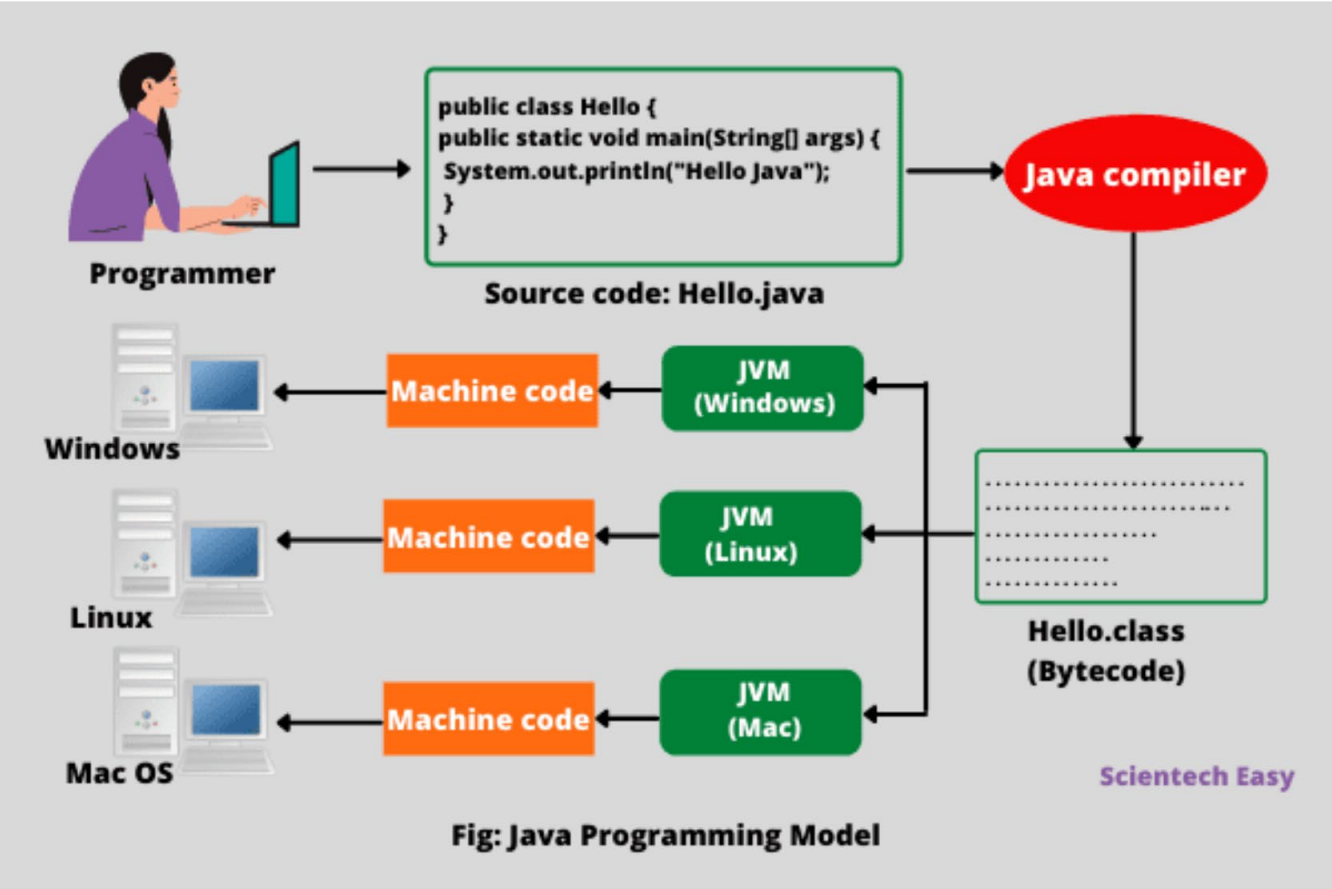
- Java was created by **Sun Microsystems team** led by Patrick Naughton and James Gosling (1991), originally named Oak (Gosling liked the look of an oak tree that was right outside his window at Sun)
- Intended to design a **small computer language** that could be used for consumer devices/appliances e.g. the cable TV switchboxes.
- Devices **do not have a lot of power or memory**, the language had to be small and generate very tight code and different manufacturers may choose different central processing units (CPUs), thus the language not be tied to any architectures.
- The team developed a **two-step translation process** to simplify the task of compiler writing for each class of appliances

Origins of Java Language

INTRODUCTION

- Significance of Java **translation process**:
  - Writing a compiler (translation program) **for each type** of appliance processor would have been **very costly**
  - Instead, developed **intermediate language** that is the same for all types of processors: Java byte-code
  - Therefore, only a small, easy to write program was needed to **translate byte-code into the machine code** for each processor
  - **Java Virtual Machine (JVM)** is a virtual runtime environment that enables Java programs to execute, regardless of the underlying hardware or operating system

Origins of Java Language



Source:https://www.scientecheasy.com/2021/03/java-bytecode.html/

JVM: Java Virtual Machine

Origins of Java Language

INTRODUCTION

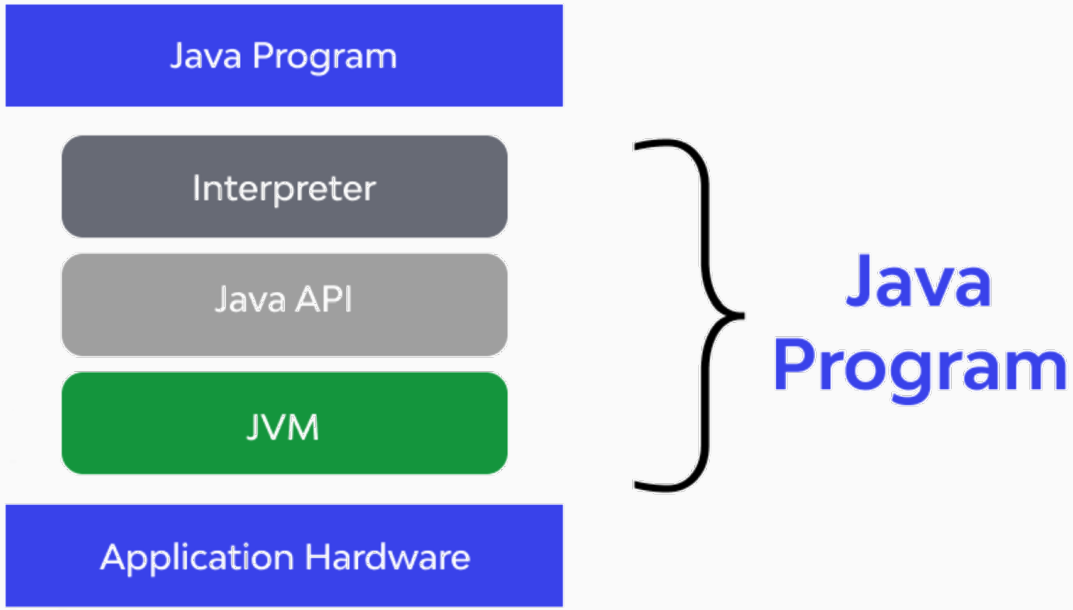
- Patrick Naughton and Jonathan Payne at Sun Microsystems developed a **Web browser** that could run programs over the Internet (1994) and evolved into the HotJava browser.
  - Beginning of Java's **connection to the Internet**
  - To show off the power of **applets**, they made the browser capable of executing code inside web pages.
- **Netscape** Incorporated made its Web browser capable of running Java programs (1995)
  - Other companies follow suit

Origins of Java Language

- Java is a full-featured, **general-purpose programming language** that is capable of developing robust mission-critical applications.
- Today, it is used for developing stand alone applications, desktops, Web programming, servers, and mobile devices
  - Run on the web browser called the **applets**
  - Used to develop applications on the server side, called **Java servlets** or Javaserver pages (JSP)
  - Used to develop applications for **small hand-held devices**, such as personal digital assistants and cell phones

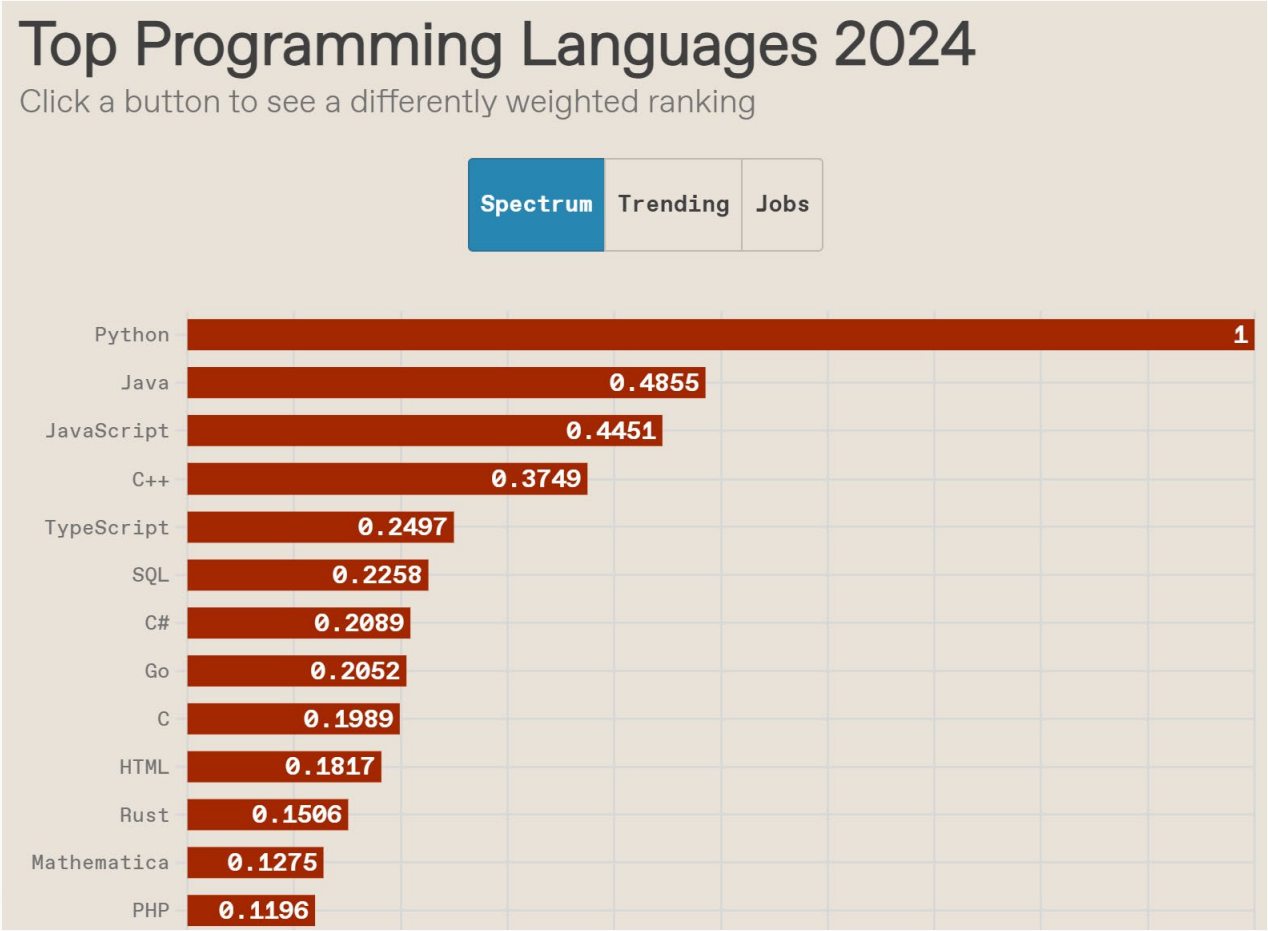
Origins of Java Language

- Java **language specification** defines the Java standard and the technical definition of the language that includes the **syntax and semantics** of the Java programming language, hence it is stable
- Java **application program interface (API)** contains predefined classes and interfaces for developing Java programs and still expanding



Source: https://www.wallarm.com/what/what-is-java-api

Java Language among the Top 5



Source: https://spectrum.ieee.org/top-programming-languages-2024

Java API

- Java 1.0 was introduced in 1995.
- December 1998, Sun announced the Java 2 platform – the brand that applies to current Java technology.
- There are **3 editions** of the Java API:
  - Java 2 standard edition **(J2SE)**: client-side standalone applications or applets
  - Java 2 Enterprise Edition **(J2EE)**: server-side applications e.g. Java servlets and JavaServer Pages
  - Java 2 Micro Edition **(J2ME)**: mobile devices, e.g. cell phones

Java Version

JAVA BASIC

Table 1-1. Evolution of the Java Language

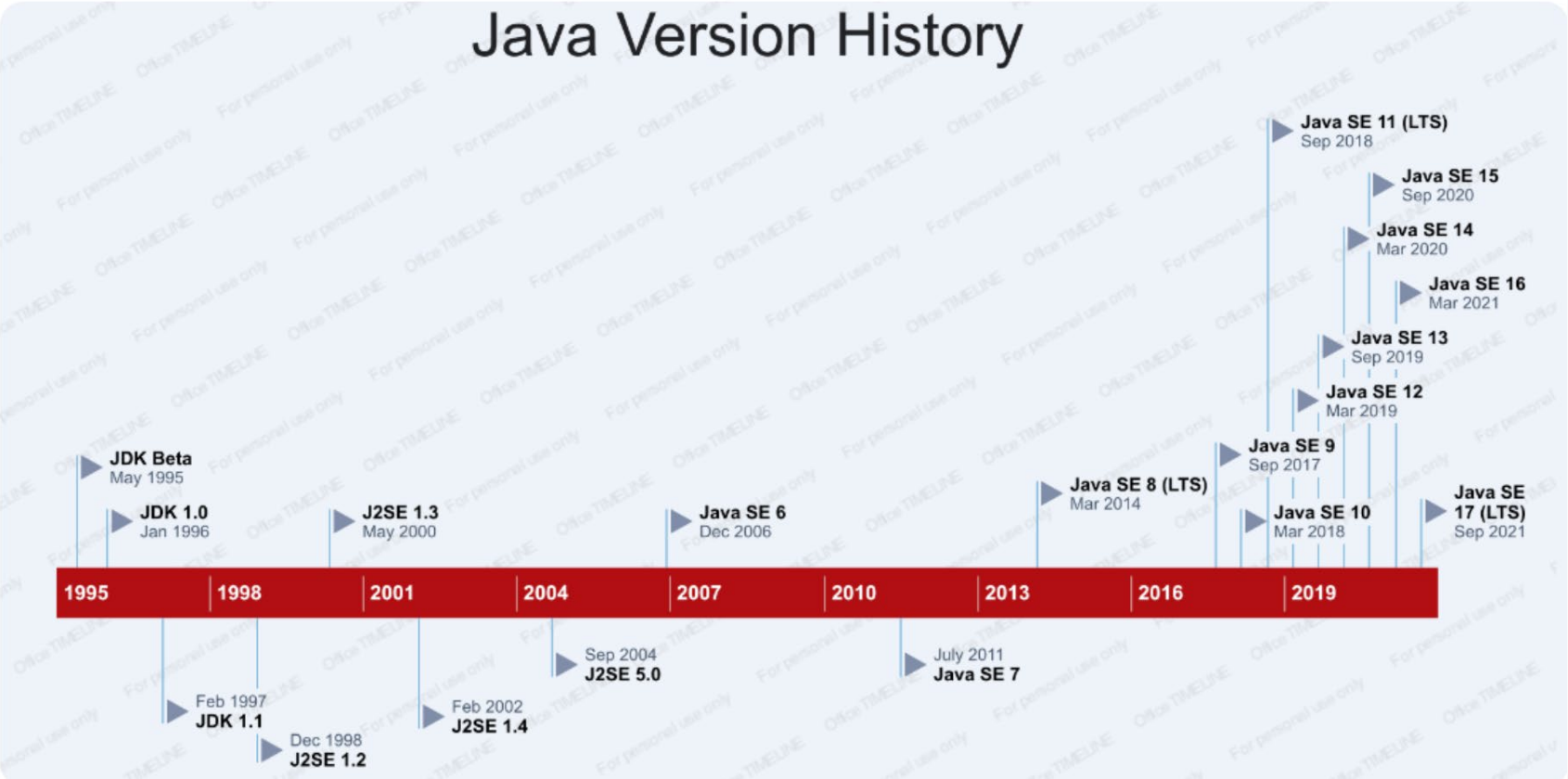
Version	New Language Features
1.0	The language itself
1.1	Inner classes
1.2	None
1.3	None
1.4	Assertions
5.0	Generic classes, "for each" loop, varargs, autoboxing, metadata, enumerations, static import

Table 1-2. Growth of the Java Standard Edition API

Version	Number of Classes and Interfaces
1.0	211
1.1	477
1.2	1524
1.3	1840
1.4	2723
5.0	3270

Java Version

JAVA BASIC



https://dzone.com/articles/alikemaltasci-the-evolution-of-java-time-based-release

Java Version

- The latest version of the Java Platform, Standard Edition (formerly known as J2SE) is **Java SE 24** (as of 2025)
- Sun releases each version of J2SE with a Java Development toolkit (JDK)
- E.g. J2SE 5.0, the JDK is called JDK 5.0 – formerly was known as JDK1.5
- For latest Java SE 24 or Java 24, it is **JDK 24**

  (source: https://www.oracle.com/my/java/technologies/downloads/)
- JDK consists of a set of separate programs for developing and testing Java programs, each of which is invoked from a command line.

Java Development Tools

- Java development tool is a software that provides an **integrated development environment (IDE)** for rapidly developing Java programs.
- Other Java development tools on the market:
  - JBuilder by Borland
  - NetBeans Open Source by Sun
  - Eclipse Open Source by IBM
- Other useful tools:
  - Code warrior by Metrowerks
  - TextPad Editor
  - JCreator LE
  - JEdit
  - JGrasp
  - BlueJ
  - DrJava

https://aglowiditsolutions.com/blog/top-java-development-tools/

Do you know the difference between Visual Studio, Visual Studio Code and VS Code? Are they different or the same?

Java Program and Java Applications

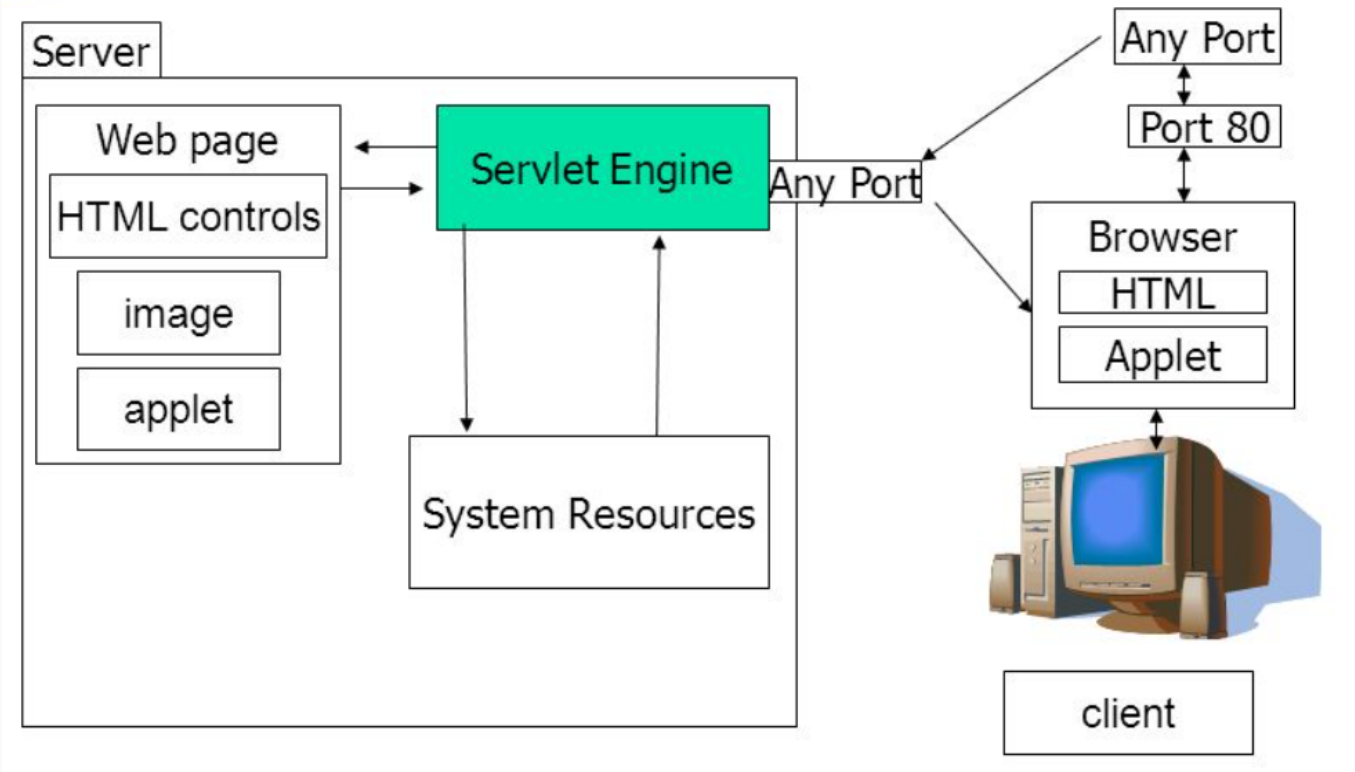
JAVA BASIC

- Java **program** can be written in many ways:
  - Applications
  - Applets
  - Servlets
- A Java **application** or "regular" Java programs are **standalone programs** that can be executed from any computer with a JVM.
- It is a **class** with a **method named main**
  - When a Java application program is run, the **run-time** system automatically invokes the method named main
  - All Java application programs **start with the main method**

Java Applets and Java Servlets

- A **Java applet** (little Java application) is a Java program that is meant to be run from a Web browser:
  - Can be run from a location on the Internet
  - Can also be run with an applet viewer program for debugging
  - Applets always use a windowing interface
- A **Java servlet** is a special kind of Java programs that run from a Web server to generate dynamic Web contents:
  - Part of Java's server-side programming
  - Run on a server, handling HTTP requests and responses
  - Interacting with databases or generating custom responses to users' requests

Java Applets and Java Servlets

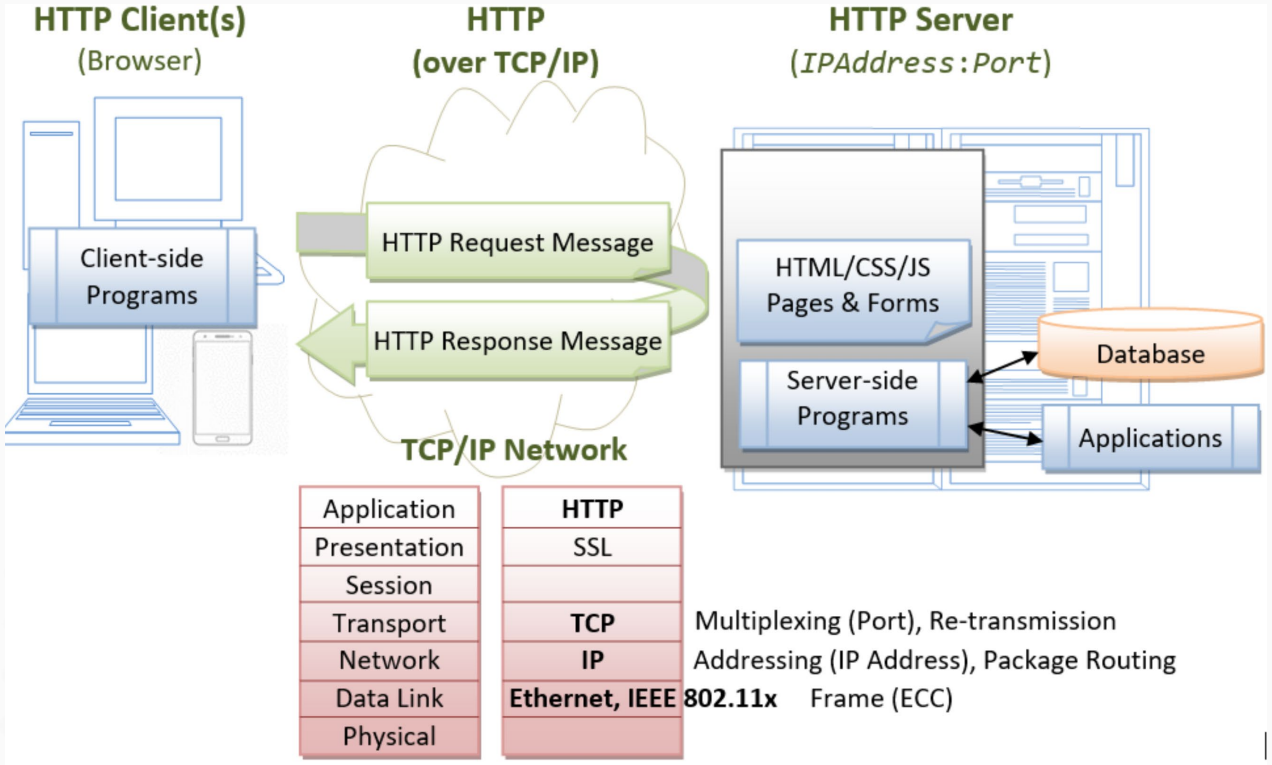


Source: https://slideplayer.com/slide/6390757/

Java Servlet is also known as Jakarta Servlet

JAVA BASIC

- **Java Servlet** technology and other Java Enterprise features were transitioned to the **Jakarta EE** platform.
- It happened when the enterprise Java specifications moved **from Oracle to the Eclipse Foundation** in 2017.

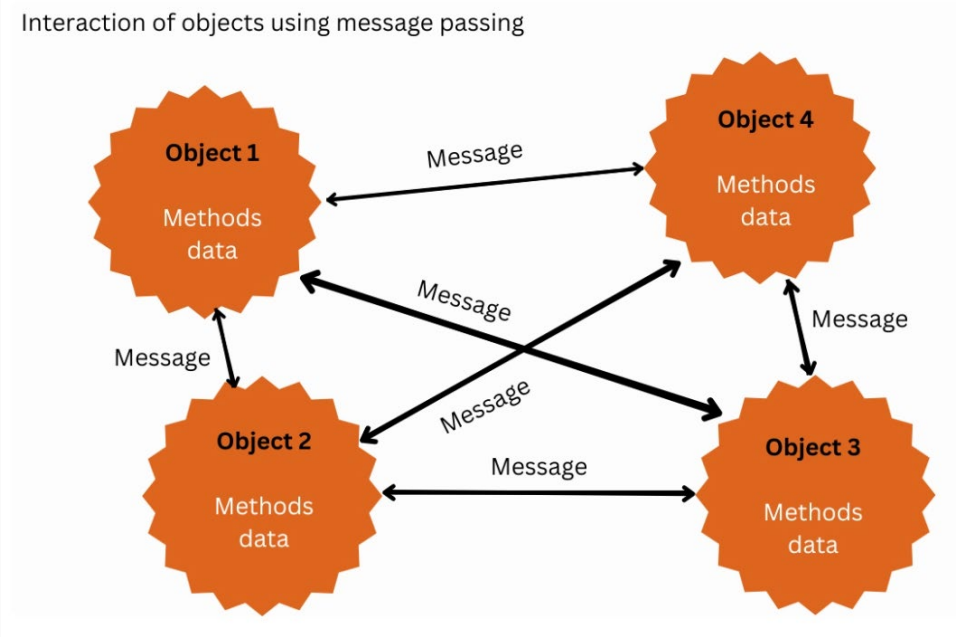


Source: https://www3.ntu.edu.sg/home/ehchua/programming/java/JavaServletCaseStudy.html

Objects and Methods in Java

JAVA BASIC

- Java is an **object-oriented programming** (OOP) language
- Programming methodology that views a program as consisting of **objects that interact** with one another by means of **actions** (called methods)
- Objects of the same kind are said to have the **same type** or be in the **same class**



Source: https://www.naukri.com/code360/library/characteristics-of-object-oriented-programming

Compiler and Java Virtual Machine (JVM)

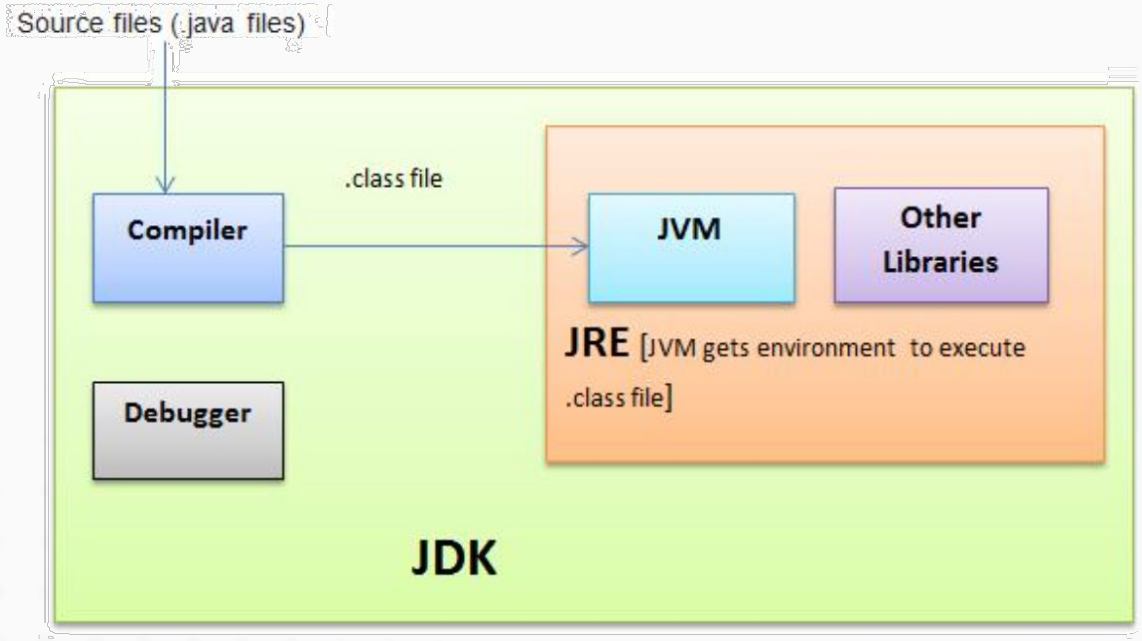
JAVA BASIC

- A programmer writes Java programming **statements** for a program known as **source code**
- A text editor is used to edit and save a Java source code file that has a **.java file extension**
- A **compiler** is a program that translates source code into an executable form, is **run** using a source code file as input
- **Syntax errors** that may be in the program will be discovered during compilation that are mistakes that the programmer has made that violate the rules of the programming language
- The **compiler** creates another file that holds the **translated instructions**

Compiler and Java Virtual Machine (JVM)

JAVA BASIC

- Byte code files end with the **.class** file extension
- The **JVM is a program** that emulates a micro-processor
- The JVM executes instructions as they are read
- JVM is often called an **interpreter**
- Java is often referred to as an **interpreted language**



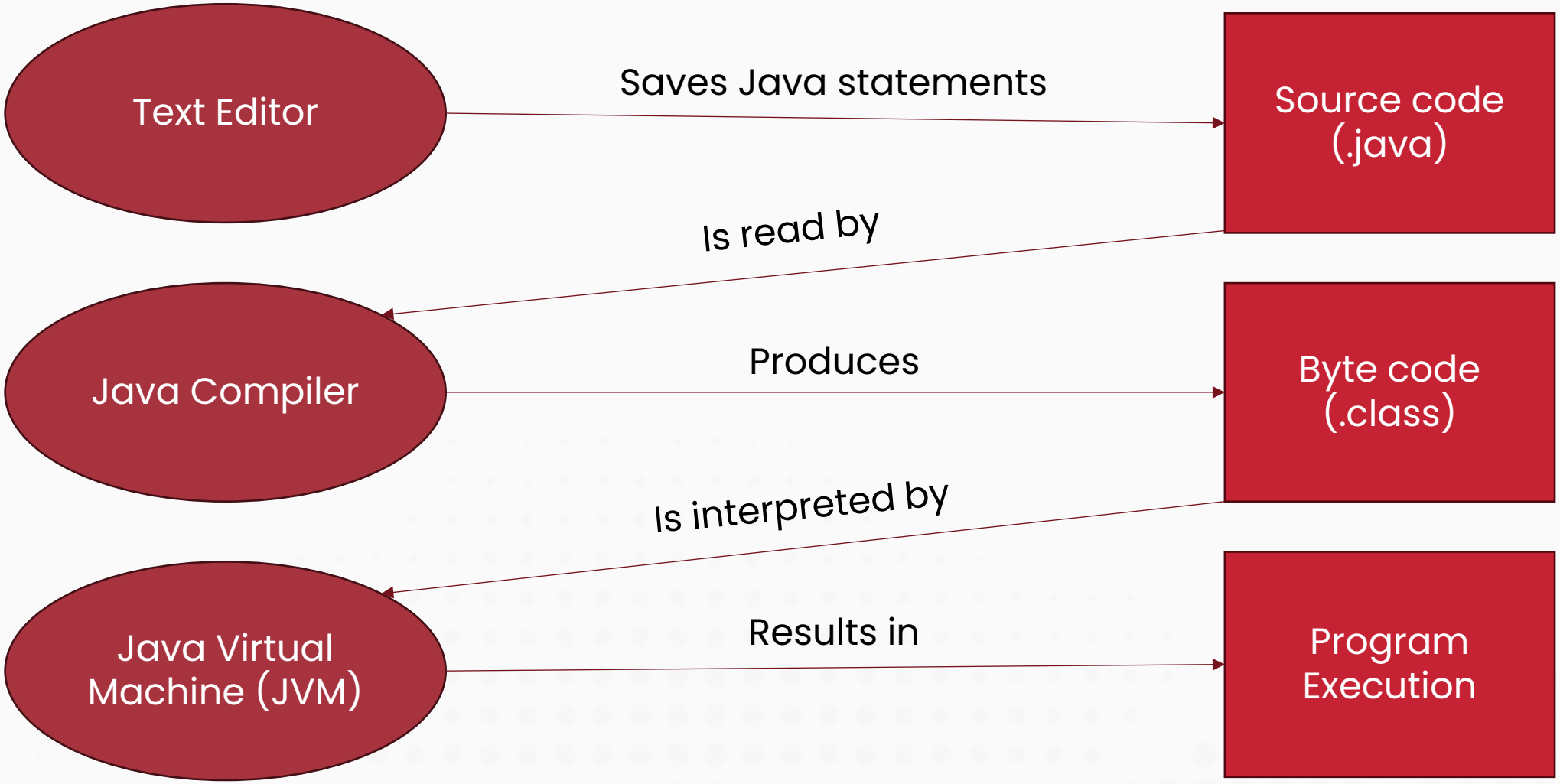
Source: https://www.startertutorials.com/corejava/java-virtual-machine.html

Byte-Code and JVM

JAVA BASIC

- The **compilers** for most programming languages **translate high-level programs** directly **into the machine language** for a particular computer
  - Since different computers have different machine languages, a different compiler is needed for each one
- In contrast, the **Java compiler translates** Java programs **into byte-code**, a machine language for a fictitious computer called the **Java Virtual Machine (JVM)**
  - Once compiled to byte-code, a Java program can be used on any computer, making it very portable

Program Development Process

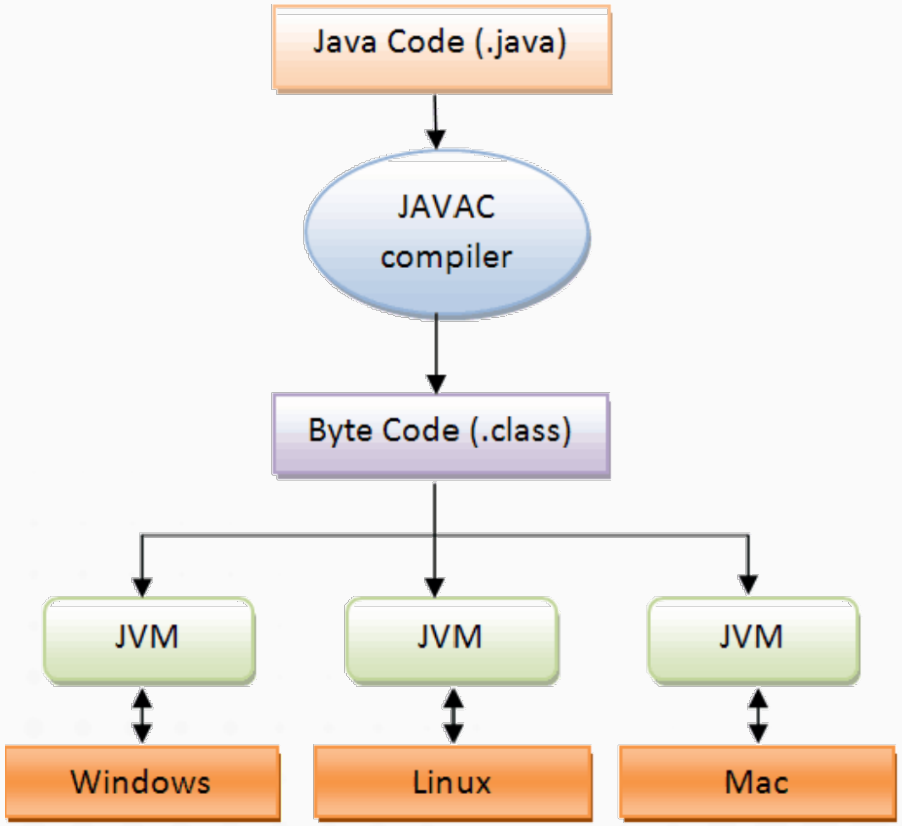


Portability

- Portable means that a program may be **written on one type of computer** and then **run on a wide variety of computers**, with little or no modification
- Java byte code **runs on the JVM** and not on any particular CPU
- Therefore, compiled Java programs are **highly portable**
- JVMs exist on many **platforms** e.g. Windows, Macintosh, Linux, Unix, BSD
- With most programming languages, portability is achieved by compiling a program for each CPU it will run on, **Java provides a JVM for each platform** so that programmers do not have to recompile for different platforms

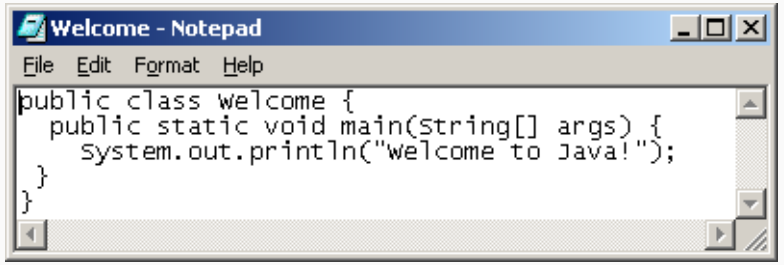
Portability

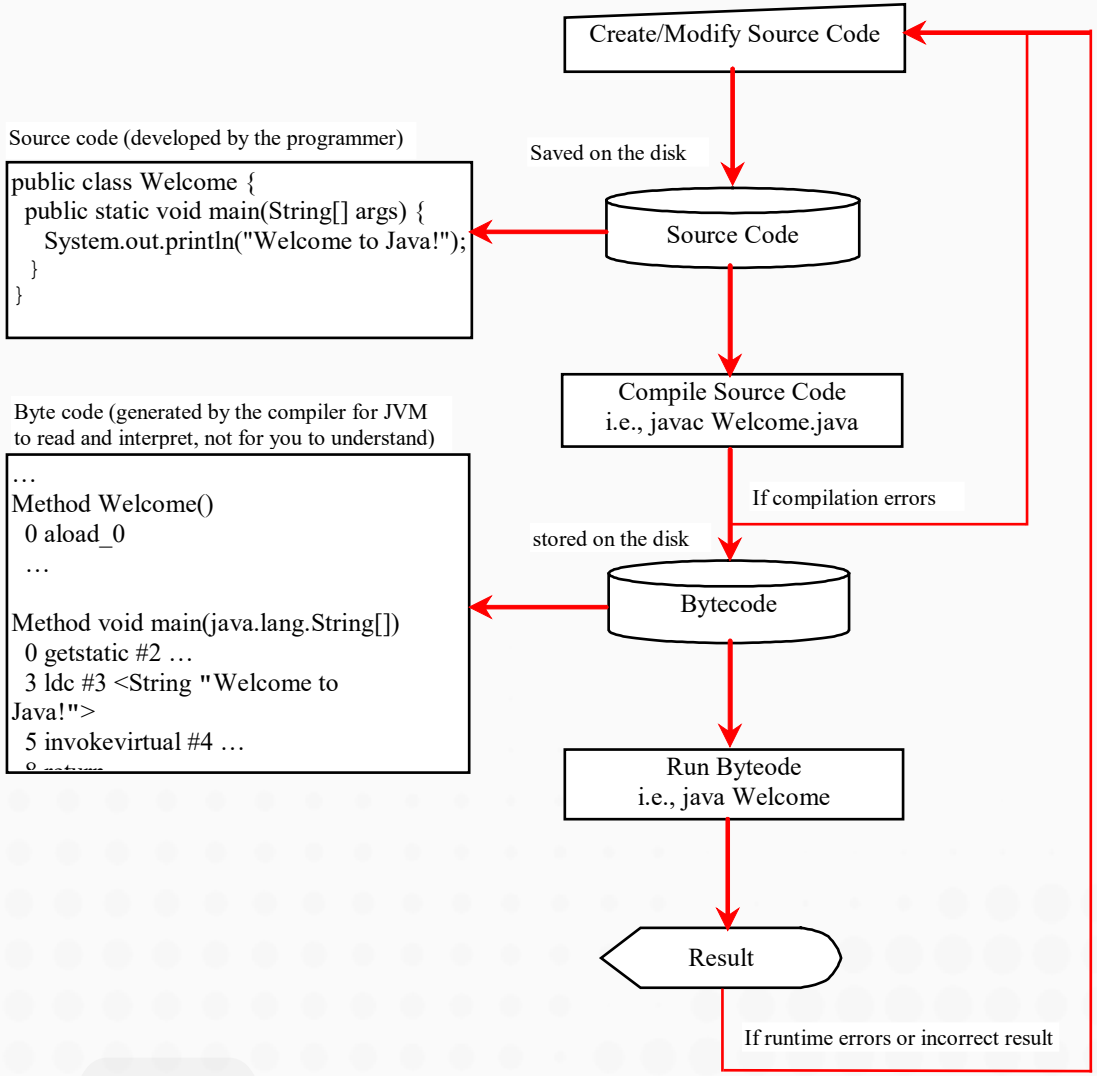
- With most programming languages, portability is achieved by compiling a program for each CPU it will run on, Java provides a **JVM for each platform**
- Thus, programmers **do not have to recompile** for different platforms



Source: https://www.viralpatel.net/java-virtual-machine-an-inside-story/

Creating, Compiling, and Running Programs





Parts of a Java Program: Source Code and Class

- A **Java source code** file contains **one or more Java classes**.
- If more than one class is in a source code file, **only one** of them may be **public**.
- The **public class** and the **filename** of the source code file **must match**
  - E.g. A class named `HelloApp` must be in a file named `HelloApp.java`
- The public class does not necessarily contain the `main` method.
- Each **Java class** can be **separated into parts**.

Parts of a Java Program: Compile and Run

- Example: `HelloApp.java`
- To **compile** the example: **`javac HelloApp.java`**
  - Notice the **.java file extension** is needed.
  - This will result in a file named `HelloApp.class` being created.
- To **run** the example: **`java HelloApp`**
  - Notice there is **no file extension** here.
  - The java command assumes the extension is .class.

Analyzing the Example

```
// This is my first Java program.
public class HelloApp
{

}
```

This is a java **comment**.
It is ignored by the compiler.

This is a **class header** for the class `HelloApp`

This area is the **body of the class** `HelloApp`.
All of the data and methods for this class will be between these **curly braces**.

Analyzing the Example

```
// This is my first Java program
public class HelloApp
{
public static void main(String[] args)
      {
      }
}
```

This is Java **main method**. Every Java application must have a main method.

This area is the body of the main method. All of the actions to be completed during the main method will be between these curly braces.

Analyzing the Example

```
// This is my first Java program.
public class HelloApp
{
public static void main(String[] args)
      {
            System.out.println("Programming great fun");
      }
}
```

This is the **Java Statement** that is executed when the program runs.

“Anatomy” of a Java Program

The "anatomy" of a Java program refers to its basic structure and **essential components/parts**:

1. Comments (Optional but recommended)
2. Keywords
3. Modifiers
4. Statements
5. Blocks
6. Classes
7. Methods
8. `main()` method
9. Package

#1: Comments and Example

- Help programmers to communicate and understand the program
- Not a programming statement, thus ignored by the compiler
- Preceded with // on a line
- Enclosed between /* and */ on one or **several lines** (see e.g. below)

```
public class HelloWorld {
    public static void main(String[] args) {
        // This is a single-line comment explaining the code below
        System.out.println("Hello, world!"); // Prints a greeting

        /*
         * This is a multi-line comment
         * It explains the purpose of the program:
         * The program greets the world!
         */
    }
}
```

#2: Keywords

- Words that have a **specific meaning to the compiler**
- Keywords in the sample program are:
  - public
  - class
  - static
  - void
  - int
  - double
  - boolean
  - continue
  - return
  - private
  - protected
  - package
- Java Language Keywords: see also https://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html
- Keywords are **lower case** (Java is a case-sensitive language)
- Keywords cannot be used as a programmer-defined identifier

Java Reserved Keywords

- abstract
- boolean
- break
- byte
- case
- catch
- char
- class
- const
- continue
- default
- do
- double
- else
- extends
- final
- finally
- float
- for
- goto
- if
- implements
- import
- instanceof
- int
- interface
- long
- native
- new
- package
- private
- protected
- public
- return
- short
- static
- strictfp
- super
- switch
- synchronized
- this
- throw
- throws
- transient
- try
- void
- volatile
- while

#3: Modifiers

- Specify the **properties of data/variables, methods, and classes** and how they can be used, example of modifies as below:
  - public ➔ data, method or class can be accessed by other classes
  - private ➔ cannot be accessed by other classes
  - protected ➔ allows members to be accessed within the same package and by subclasses, even if they are in different packages
  - final ➔ introduces restrictions or prevents modification
  - static ➔ belongs to the class itself rather than to any instance of the class
  - abstract ➔ abstract class cannot be instantiated, abstract method has no body (just a declaration)

#4: Statements

- Represents an **action** or a sequence of actions
- Example of statement:

```
System.out.println("Welcome to Java!");
```

  is a statement to display the greeting "Welcome to Java!"
- Every statement in Java ends with a semicolon **;**

#5: Blocks

- Groups the components of the program using the **open and close braces { … }** in the program.
- Every class has a **class block** that groups the data and the methods of the class.
- Every method has a **method block** that groups the data and the methods of the class.
- Block may be **nested**, meaning that one block can be placed within another.

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Class block

Method block

#6: Classes

- Class is an **essential** Java construct
- Classes are central to Java
- Programming in Java consists of defining a **number of classes**:
  - Every program is a class (A program is defined by using one or more classes)
  - All programmer-defined types are classes

Classes: Example 1 and 2

ANATOMY OF JAVA PROGRAM

Example 1: Program named ClassA.java with main method

```
public class ClassA {
    public static void main (String[] args) {
      System.out.println ("Try your best");
    }
}
```

Example 2: Program named **ClassA.java** has **two classes** i.e. ClassA and ClassB

```
public class ClassA {
  private int yearborn=2003;
  public String methodA() { return "Aim High";  }
  public int getYearBorn() { return yearborn; }
}

class ClassB {
    public static void main (String[] args) {
      ClassA obj1 = new ClassA();
      System.out.println (“Your age: ” + (2025 - obj1.getYearBorn()));
      System.out.println (“My message: ” + obj1.methodA());
    }
}
```

Note: Without a modifier e.g. public or private, the default access modifier is *package-private*.

#7: Methods

- A **collection of statements** that performs a sequence of operations
- A method is contained in a class
- If a method is intended to be used to **communicate with or pass information to an object**, it should be declared **public**.
- Example: Method `println()` is an **instance** method that belongs to an object instance and is applied to an object: `System.out`

Methods: Example

getYearBorn **is a class method in** ClassA. public **modifier indicates it can be accessed from anywhere.** int **indicates it return a value of type integer.**

methodA **is a class method in** ClassA. public **modifier indicates it can be accessed from anywhere.** String **indicates it return a value of String.**

```
public class ClassA {
  private int yearborn=2003;
  public String methodA() { return "Aim High";  }
  public int getYearBorn() { return yearborn; }
}

class ClassB {
    public static void main (String[] args) {
      ClassA obj1 = new ClassA();
      System.out.println ("Your age: " + (2025 - obj1.getYearBorn()));
      System.out.println ("My message: " + obj1.methodA());
    }
}
```

main **method is in** ClassB

getYearBorn **method is invoked from instance of the class,** obj1.

methodA **method is invoked from instance of the class,** obj1.

`main` Methods

Keyword `void` indicates the data type returned from this method is nothing or no value.

This is the parameter of the `main` method. It takes arguments of an array of Strings. The data type `String` starts with an upper case `S`. The square brackets indicate an array.

- Every Java application must have a main method that is declared in the following way:

```
public class ClassName
        {
                public static void main(String[] args) {
                // Statements;
        }
}
```

This method is `public`, i.e. visible from anywhere that can see this class.

The `main` method in Java is always `static`, meaning that this method can be run without creating an instance of the class.

Command-Line Arguments

```
C:\norazah> javac Greetings.java
C:\norazah> java Greetings Aqilah Ahmad
Hello, Aqilah Ahmad
```

Command-line arguments are passed to main as an array of Strings.

```
public class Greetings
{
    public static void main(String[] args)
    {
        String firstName = args[0];
        String lastName = args[1];
        System.out.println("Hello, " + firstName + "  " + lastName);
    }
}
```

#9: Package

- Java is a package-centric language
- For **good organization** and name scoping, put all classes into packages.
- A class with **default access** can be seen only by classes within the same package, also known as ***package-private* access level**
- If class A and class B are in **different packages**, and **class A has default access**, class B will not be able to create an instance of class A, or even declare a variable or return type of class A.

Package: Example

Class Sludge and class Goo are both in different packages

Note: Class Sludge should be public to enable class Goo in different package accessing the method `testIt()`

```
package cert;
public class Sludge {
  public void testIt() {
    System.out.println("sludge");
  }
}
```

```
package book;
import cert.*; // Import all classes in the cert
package class Goo {
  public static void main(String[] args) {
    Sludge o = new Sludge();
    o.testIt();
  }
}
```

Source File Declaration Rules

- A **source code file** can have only **one public class** but can have more than one non-public class
- If the source file contains a public class, the **filename must match** the public class name.
- A file can have **only one package statement**, but **multiple imports**.
- The **package statement** (if any) **must be the first** (non-comment) line in a source file, while the **import statements** (if any) must come after the package and before the class declaration.
- If there is no package statement, import statements must be the first (non-comment) statements in the source file.
- Package and import statements apply to all classes in the file.
- Files with **no public classes** can have a name that does not match any of the classes in the file (the main method is in another file)

Libraries

- Java programs are usually not written from scratch
- There are hundreds of **library classes** for all occasions
- Library classes are **organized into packages**, example:
  - java.util — miscellaneous utility classes
  - java.awt — windowing and graphics toolkit
  - javax.swing — GUI development package

```
import java.util.ArrayList;

public class Example {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        System.out.println("Numbers: " + numbers);
    }
}
```

Output:

```
Numbers: [10, 20, 30]
```

`Import` Statement

- Full library class names include the package name, example:

  java.awt. Color

  javax.swing.JButton

- The `import` statement at the top of the source file enable programmers to refer to library classes by their short names:

```
import  javax.swing.JButton;
...
    JButton go = new JButton("Go");
```

Fully-qualified name

`Import` Statement Using a Wildcard

ANATOMY OF JAVA PROGRAM

- Import names for all the classes in a package by using a **wildcard .*** for example:

```
import    java.awt.*;
import    java.awt.event.*;
import    javax.swing.*;
```

Imports all classes from awt, awt.event, and swing packages

- The package `java.lang` is **imported automatically** into all classes that defines System, Math, Object, String, and other commonly used classes.

“Anatomy” of Java Program: Exercise – Identify 9 parts

ANATOMY OF JAVA PROGRAM

```java
// Package declaration (optional)
package mypackage;

// Import statements (optional)
import java.util.Scanner;

// Class declaration
public class MyProgram {
    // main() method - entry point
    public static void main(String[] args) {
        // Print a message
        System.out.println("Hello, Java!");
    }
}
```

“Anatomy” of a Java Program

The "anatomy" of a Java program refers to its basic structure and **essential components/parts**:

1. Comments (Optional but recommended)
2. Keywords
3. Modifiers
4. Statements
5. Blocks
6. Classes
7. Methods
8. `main()` method
9. Package